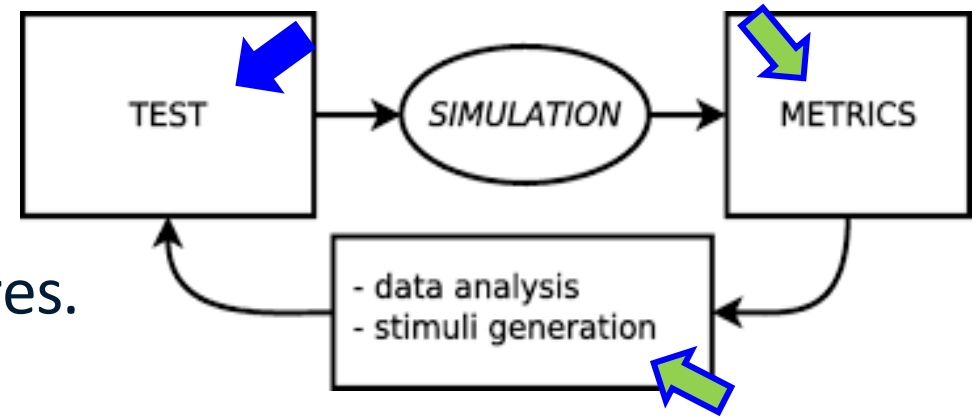
More on Functional Coverage and CRV in Verification

EEL 4712 Spring 2026

Design Verification: Testing and Coverage



Directed Testing, Constrained-Random Testing

- **Directed tests:** Explicitly define inputs to test specific scenarios to verify certain design features.
- **Random, constrained-random tests**:
  - **Random:** generate a large number of tests with random inputs
  - **Constrained-random:** random, but within user-defined constraints to uncover corner cases and bugs that might be missed by directed or completely random tests.

Coverage:

- a *simulation metric* used to measure the *progress* and *completeness* (quality) of the verification process
  - To provide feedback to improve test

Covergroups, Coverpoints, and Bins

Covergroup:

SystemVerilog construct that groups coverpoint objects of interest

```
covergroup <name> @(<sampling_event>)
                        //  (e.g., @(posedge clk))
   <label>: coverpoint <coverpoint name> {bins ... }>;
   <label>: coverpoint <coverpoint name> {bins ... }>;
     ...
endgroup;
```



```
logic [7:0] mem_addr;
logic [7:0] mem_data;
logic wr_en;
covergroup mem_inputs @(posedge clk)
  cpaddr: coverpoint mem_addr {
      bins b1 {0, 63, 127, 255};   // individual values
      bins b2 {[1:62], [64:128]}; }    // range
  cpdata: coverpoint mem_data {
        bins b1 {0, 7, [32:63]};     // indiv. & range
        bins b2 {[1:62], [128:$]; }   // $ is max
                                  // 255 in this example
  cpen: coverpoint wr_en  {bins b1 = {1}; }
                                  // single bin for wr_en = 1
endgroup
```

Covergroups, Coverpoints, and Bins

Coverpoint:

A signal, variable, or expression of interest to be sampled and monitored.

```
<label> : coverpoint <coverpoint name>
                        [ iff (condition) ]
  {
      // bins
      // options
  }
```

```
logic [7:0] mem_addr;
logic [7:0] mem_data;
logic wr_en;
covergroup mem_inputs @(posedge clk)
  cpaddr: coverpoint mem_addr {
      bins b1 {0, 63, 127, 255};   // individual values
      bins b2 {[1:62], [64:128]}; }   // range
  cpdata: coverpoint mem_data {
          bins b1 {0, 7, [32:63]};     // indiv. & range
          bins b2 {[1:62], [128:$]; }   // $ is max
                                  // 255 in this example
  cpen: coverpoint wr_en  {bins b1 = {1}; }
                                  // single bin for wr_en = 1
endgroup
```

Covergroups, Coverpoints, and Bins

Bins: Use to define specific values or ranges of values for a coverage point that needs to be monitor.

- **Scalar** bins: results in a single bin for all values/ranges specified in bin definition
  - **Syntax**: **bins bin_name = {value, [start:end]};**
  - **Example**: cpen: coverpoint wr_en { bins b1 = {1} }; // single bin for wr_en = 1
  - **Example**: cpaddr: coverpoint mem_addr { bins b2 {[1:62], [64:128]} };
    // creates 1 bin for all the values or ranges specified in bin definition
- **Vector** bins: results in one bin for each of the values orranges specified in bin definition
  - **Syntax**: **bins bin_range [] = {value, [start:end]};**
  - **Example**: cpaddr: coverpoint mem_addr []{ bins b2 {[1:62], [64:128]} };
    // creates 2 bins, one for [1:62], one for [64:128]
- Bins with a **condition**
  - **Syntax**: **bins bin_name = {[start:end]} with (condition);**
  - **Example**: bins inputb = {[1000:2000]} with (inputb % 3 == 0); // inputb is multiple of 3

Covergroups, Coverpoints, and Bins

Bins: Use to define specific values or ranges of values for a coverage point that needs to be monitor.

- **Scalar** bins: results in a single bin for all valu
  - **Syntax**: **bins bin_name = {value, [start:end**
  - **Example**: cpen: coverpoint wr_en { bins b
  - **Example**: cpaddr: coverpoint mem_addr {
    // creates 1 bin for all the value
- **Vector** bins: results in one bin for each of the
  - **Syntax**: **bins bin_range [] = {value, [start:e**
  - **Example**: cpaddr: coverpoint mem_addr [
    // creates 2 bins, one for [1:62],
- Bins with a **condition**
  - **Syntax**: **bins bin_name = {[start:end]} with**
  - **Example**: bins inputb = {[1000:2000]} with

```
logic [7:0] mem_addr;
logic [7:0] mem_data;
logic wr_en;
covergroup mem_inputs @(posedge clk)
  cpaddr: coverpoint mem_addr {
      bins b1 {0, 63, 127, 255};   // individual values
      bins b2 {[1:62], [64:128]}; }    // range
  cpdata: coverpoint mem_data {
        bins b1 {0, 7, [32:63]};     // indiv. & range
        bins b2 {[1:62], [128:$]; }   // $ is max
                                      // 255 in this example
  cpen: coverpoint wr_en  {bins b1 = {1}; }
                                // single bin for wr_en = 1
endgroup
```

cross Between Coverpoints

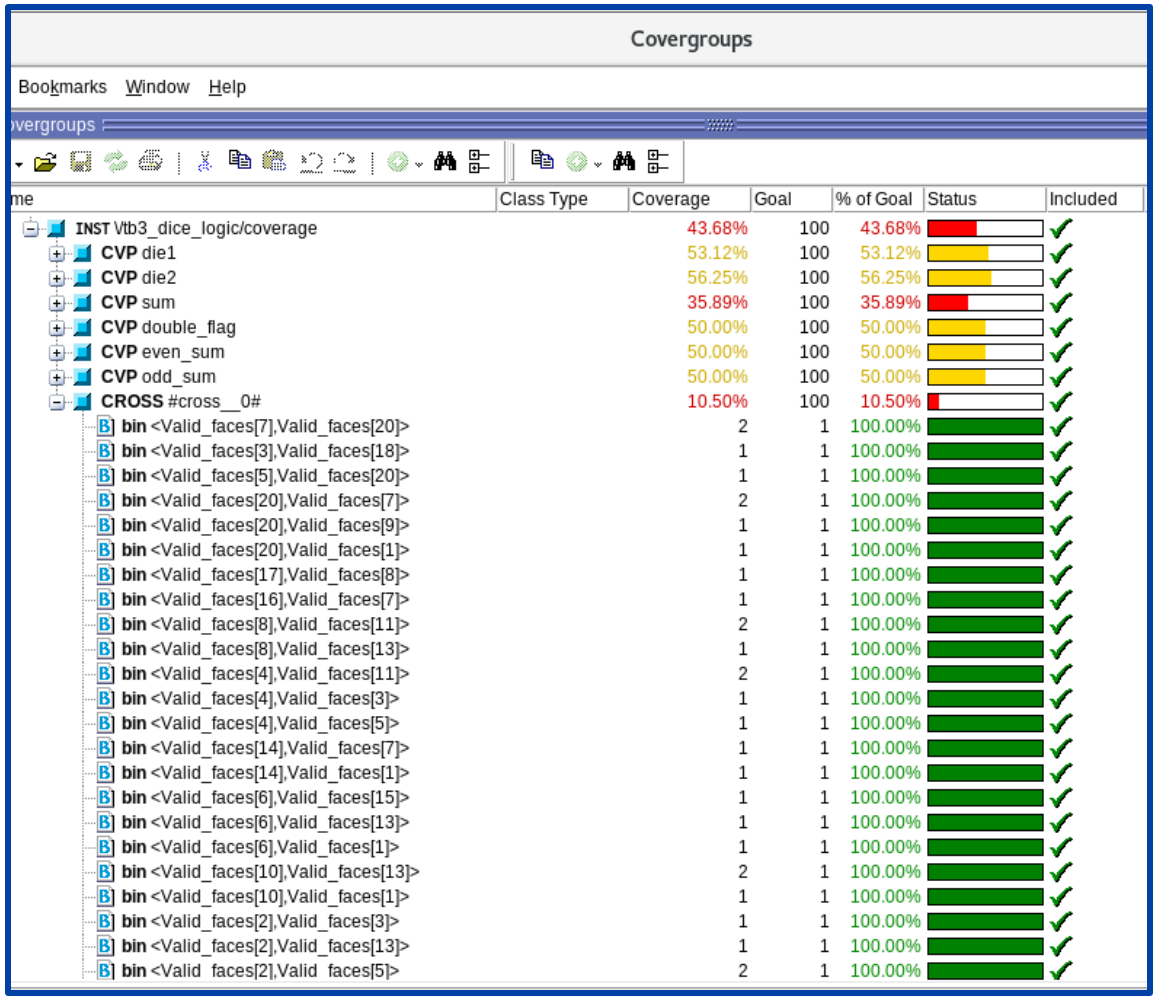
Measures **combinations of values** that occur across **two or more coverage points**

```
covergroup <covergroup_name>;
    coverpoint <variable1>;
    coverpoint <variable2>;
        ...
    cross <variable1>, <variable2>, ... ;
endgroup
```

(Example from dice_tb3.sv)

```
cross die1, die2 {
  // Optional: restrict to valid combinations only
    ignore_bins invalid_pairs =
        binsof(die1.Invalid_faces) ||
        binsof(die2.Invalid_faces);
}
```

dice_tb3.sv
Coverage Window



* **ignore_bins** keyword defines which values or value ranges you do not want to track

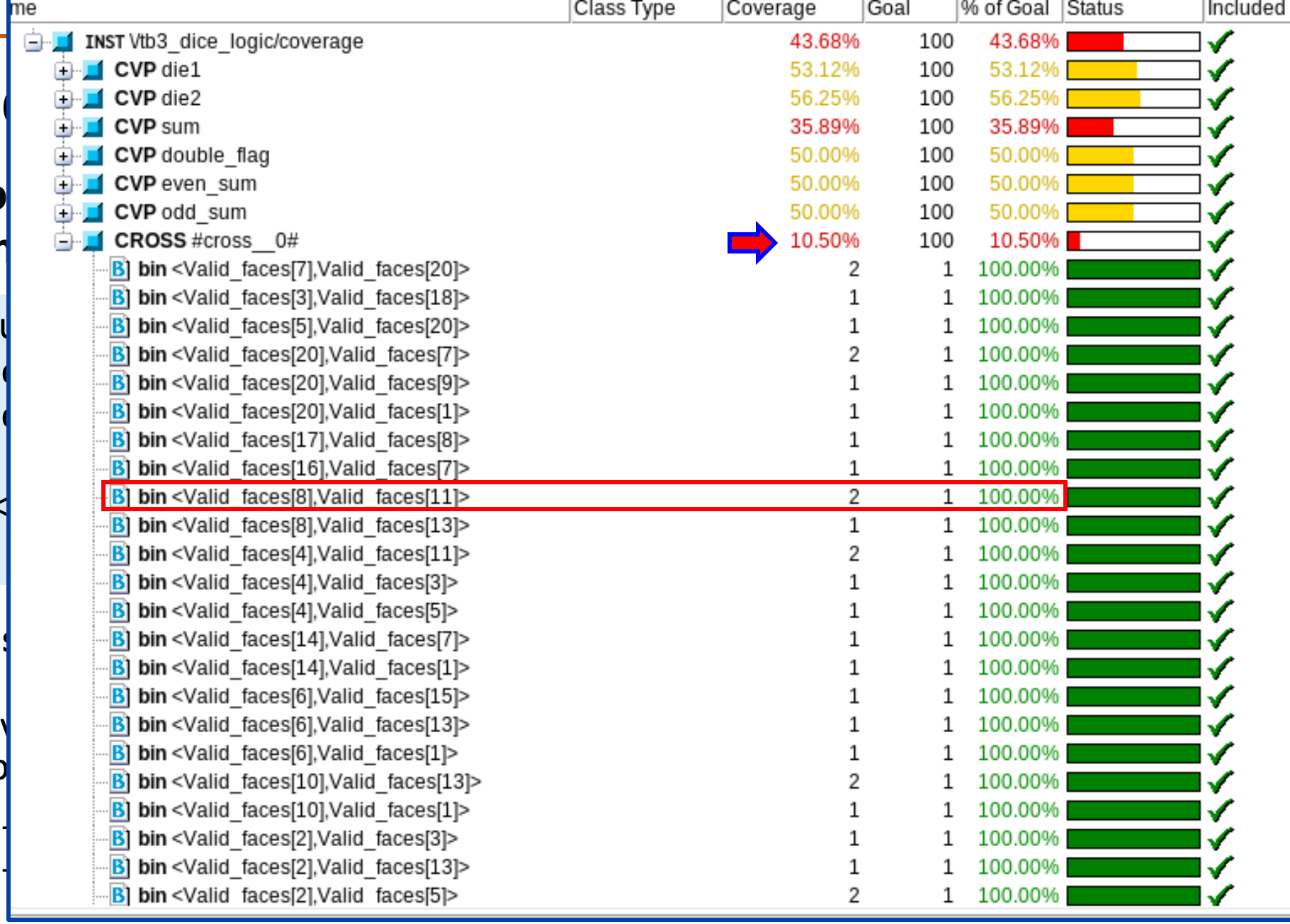
cross Betwe

Measures **combinatio**
occur across **two or m**

```
covergroup <covergrou
    coverpoint <variabl
    coverpoint <variabl
        ...
    cross <variable1>, <
endgroup
```

(Example from dice_tb3.

```
cross die1, die2 {
  // Optional: restrict to v
    ignore_bins invalid_p
        binsof(die1.Invalid_
        binsof(die2.Invalid_
}
```

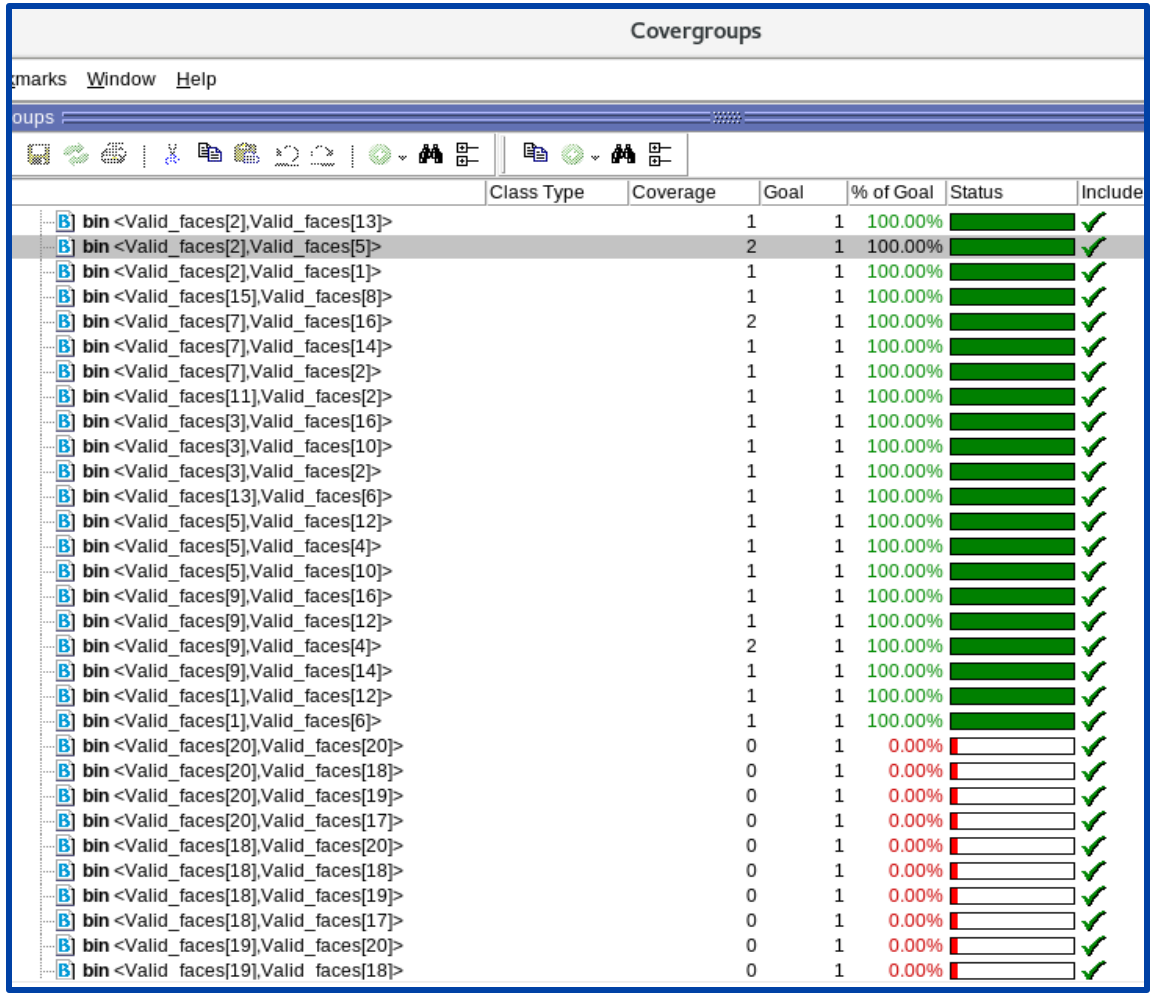


me	Class Type	Coverage	Goal	% of Goal	Status	Included
INST \tb3_dice_logic/coverage		43.68%	100	43.68%		
CVP die1		53.12%	100	53.12%		
CVP die2		56.25%	100	56.25%		
CVP sum		35.89%	100	35.89%		
CVP double_flag		50.00%	100	50.00%		
CVP even_sum		50.00%	100	50.00%		
CVP odd_sum		50.00%	100	50.00%		
CROSS #cross__0#		10.50%	100	10.50%		
bin <Valid_faces[7],Valid_faces[20]>		2	1	100.00%		
bin <Valid_faces[3],Valid_faces[18]>		1	1	100.00%		
bin <Valid_faces[5],Valid_faces[20]>		1	1	100.00%		
bin <Valid_faces[20],Valid_faces[7]>		2	1	100.00%		
bin <Valid_faces[20],Valid_faces[9]>		1	1	100.00%		
bin <Valid_faces[20],Valid_faces[1]>		1	1	100.00%		
bin <Valid_faces[17],Valid_faces[8]>		1	1	100.00%		
bin <Valid_faces[16],Valid_faces[7]>		1	1	100.00%		
bin <Valid_faces[8],Valid_faces[11]>		2	1	100.00%		
bin <Valid_faces[8],Valid_faces[13]>		1	1	100.00%		
bin <Valid_faces[4],Valid_faces[11]>		2	1	100.00%		
bin <Valid_faces[4],Valid_faces[3]>		1	1	100.00%		
bin <Valid_faces[4],Valid_faces[5]>		1	1	100.00%		
bin <Valid_faces[14],Valid_faces[7]>		1	1	100.00%		
bin <Valid_faces[14],Valid_faces[1]>		1	1	100.00%		
bin <Valid_faces[6],Valid_faces[15]>		1	1	100.00%		
bin <Valid_faces[6],Valid_faces[13]>		1	1	100.00%		
bin <Valid_faces[6],Valid_faces[1]>		1	1	100.00%		
bin <Valid_faces[10],Valid_faces[13]>		2	1	100.00%		
bin <Valid_faces[10],Valid_faces[1]>		1	1	100.00%		
bin <Valid_faces[2],Valid_faces[3]>		1	1	100.00%		
bin <Valid_faces[2],Valid_faces[13]>		1	1	100.00%		
bin <Valid_faces[2],Valid_faces[5]>		2	1	100.00%		

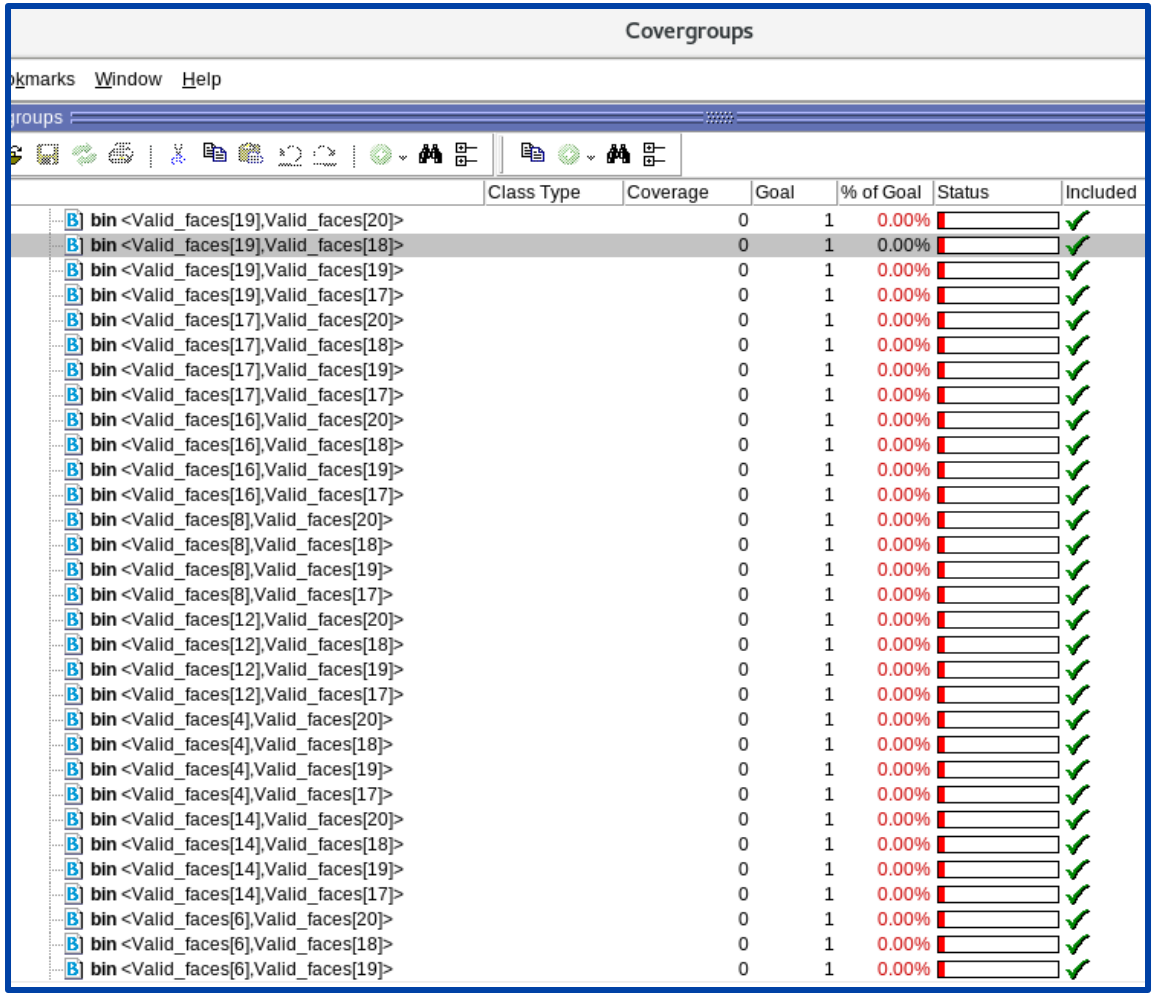
* **ignore_bins** keyword defines which values or value ranges you do not want to track

cross Keyword in Constraints (continued)

dice_tb3.sv Transcript Window

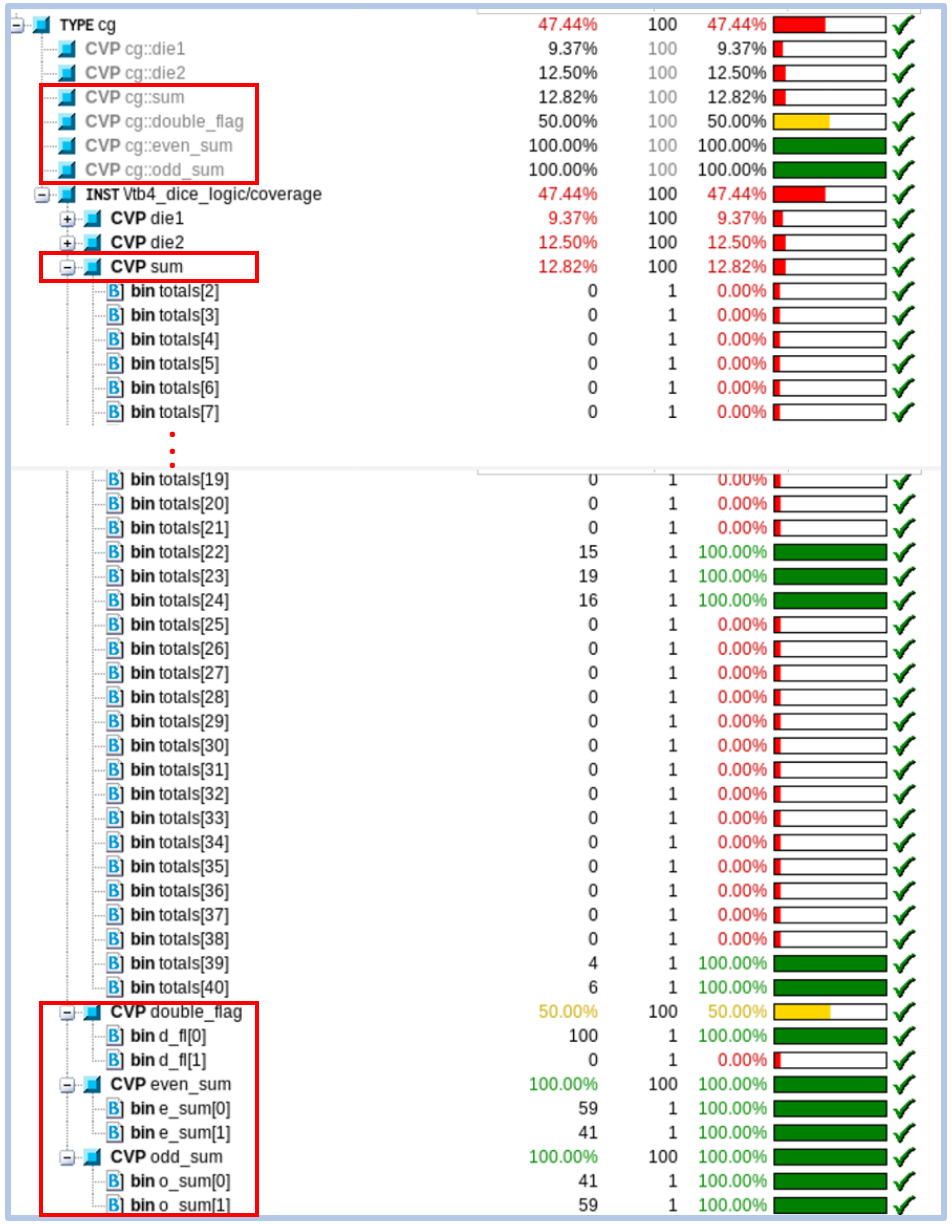


dice_tb3.sv Transcript Window

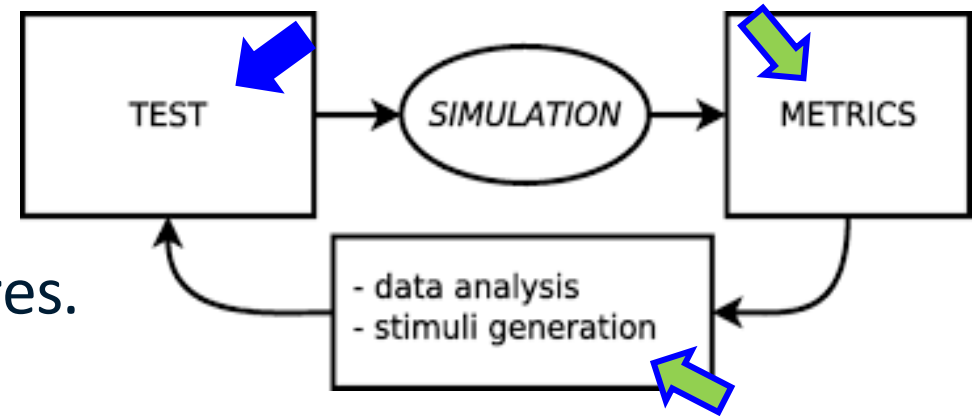


Constraints Are Not Just For Inputs

```
covergroup cg @(posedge clk);
      coverpoint die1 {
            bins Valid_faces[]   = {[1:20]};  // Valid range
            bins Invalid_faces[] = {0,[21:31]}; // Invalid range
      }
      coverpoint die2 {
            bins Valid_faces[]   = {[1:20]};      // Valid range
            bins Invalid_faces[] = {0,[21:31]};   // Invalid values
      }
      coverpoint sum      { bins totals[] = {[2:40]}; }
      coverpoint double_flag { bins d_fl [] = {[0:1]}; }
      coverpoint even_sum  { bins e_sum [] = {[0:1]}; }
      coverpoint odd_sum  { bins o_sum  [] = {[0:1]}; }
endgroup
```



Design Verification: Testing and Coverage



Directed Testing, Constrained-Random Testing

- **Directed tests:** Explicitly define inputs to test specific scenarios to verify certain design features.
- **Random, constrained-random tests**:
  - **Random:** generate a large number of tests with random inputs
  - **Constrained-random:** random, but within user-defined constraints to uncover corner cases and bugs that might be missed by directed or completely random tests.

Coverage:

- a *simulation metric* used to measure the *progress* and *completeness* (quality) of the verification process
  - To provide feedback to improve test

Virtual Dice Example

```
module dice_logic (
    input  logic [4:0] die1,       // Valid inputs 1-20, invalid 0, 21-31
    input  logic [4:0] die2,       // Valid inputs 1-20, invalid 0, 21-31
    output logic [5:0] sum,          // Sum of die1 and die2
    output logic double_flag,      // 1 if both dice equal
    output logic odd_sum,          // 1 if sum is odd
    output logic even_sum          // 1 if sum is even
);
  always_comb begin  // 4 "major" functions to be tested
    sum        = die1 + die2;    // represents "major" function 1
    double_flag = (die1 == die2);   // represents "major" function 2
    odd_sum    = (sum % 2);   // represents "major" function 3
    even_sum   = ~odd_sum;   // represents "major" function 4
  end
endmodule
```



CRV in SV (SystemVerilog)

Constraints is used in SV to specify how to *randomize values* of a variable to meet *specific design requirements*

Keywords:

- **inside:** Constrains a variable to take a value from a given set of values or a specified range.
- **dist:** specifies weighted or proportional probability for a random variable.
- **unique:** generated values for a random variable must be unique.
- **implication:** implies operator -> creates an if-then conditional relationship.
- **if-else:** creates an if-then-else conditional relationship.
- **solve-before:** Use to generate one variable before another, especially when one variable's value depends on another.

inside Keyword in Constraints

Constrains a variable to take a value from **a given set of values** or **a specified range.**

```
constraint <constraint_name> {
      <variable> inside { <value1, [ <start_value> : <end_value>], value2, ...};  }
```

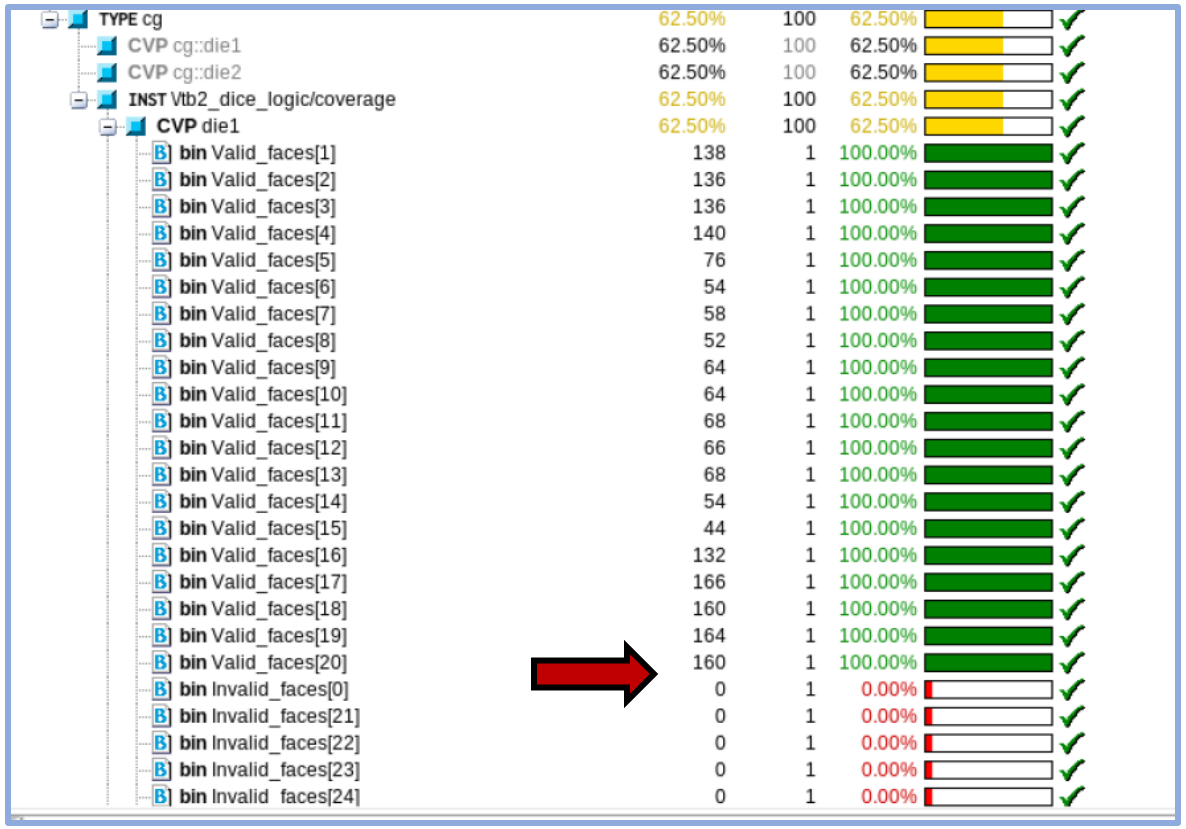
(Example from dice_tb2.sv)

```
constraint legit_c {
      die1_val inside {[1:20]};
      die2_val inside {[1:20]}; }
```

From Questa Transcript Window

```
# DIE1=14  DIE2=17  SUM=31  DOUBLE=0  ODD=1 EVEN=0
# DIE1=17  DIE2=20  SUM=37  DOUBLE=0  ODD=1 EVEN=0
# DIE1=20  DIE2=4  SUM=24  DOUBLE=0  ODD=0 EVEN=1
# DIE1=5  DIE2=8  SUM=13  DOUBLE=0  ODD=1 EVEN=0
# DIE1=11  DIE2=19  SUM=30  DOUBLE=0  ODD=0 EVEN=1
# DIE1=14  DIE2=18  SUM=32  DOUBLE=0  ODD=0 EVEN=1
# DIE1=2  DIE2=4  SUM=6  DOUBLE=0  ODD=0 EVEN=1
# DIE1=9  DIE2=20  SUM=29  DOUBLE=0  ODD=1 EVEN=0
# DIE1=3  DIE2=14  SUM=17  DOUBLE=0  ODD=1 EVEN=0
# DIE1=11  DIE2=10  SUM=21  DOUBLE=0  ODD=1 EVEN=0
# DIE1=4  DIE2=15  SUM=19  DOUBLE=0  ODD=1 EVEN=0
# DIE1=3  DIE2=18  SUM=21  DOUBLE=0  ODD=1 EVEN=0
# DIE1=5  DIE2=19  SUM=24  DOUBLE=0  ODD=0 EVEN=1
```

Coverage Window



dist Keyword in Constraints

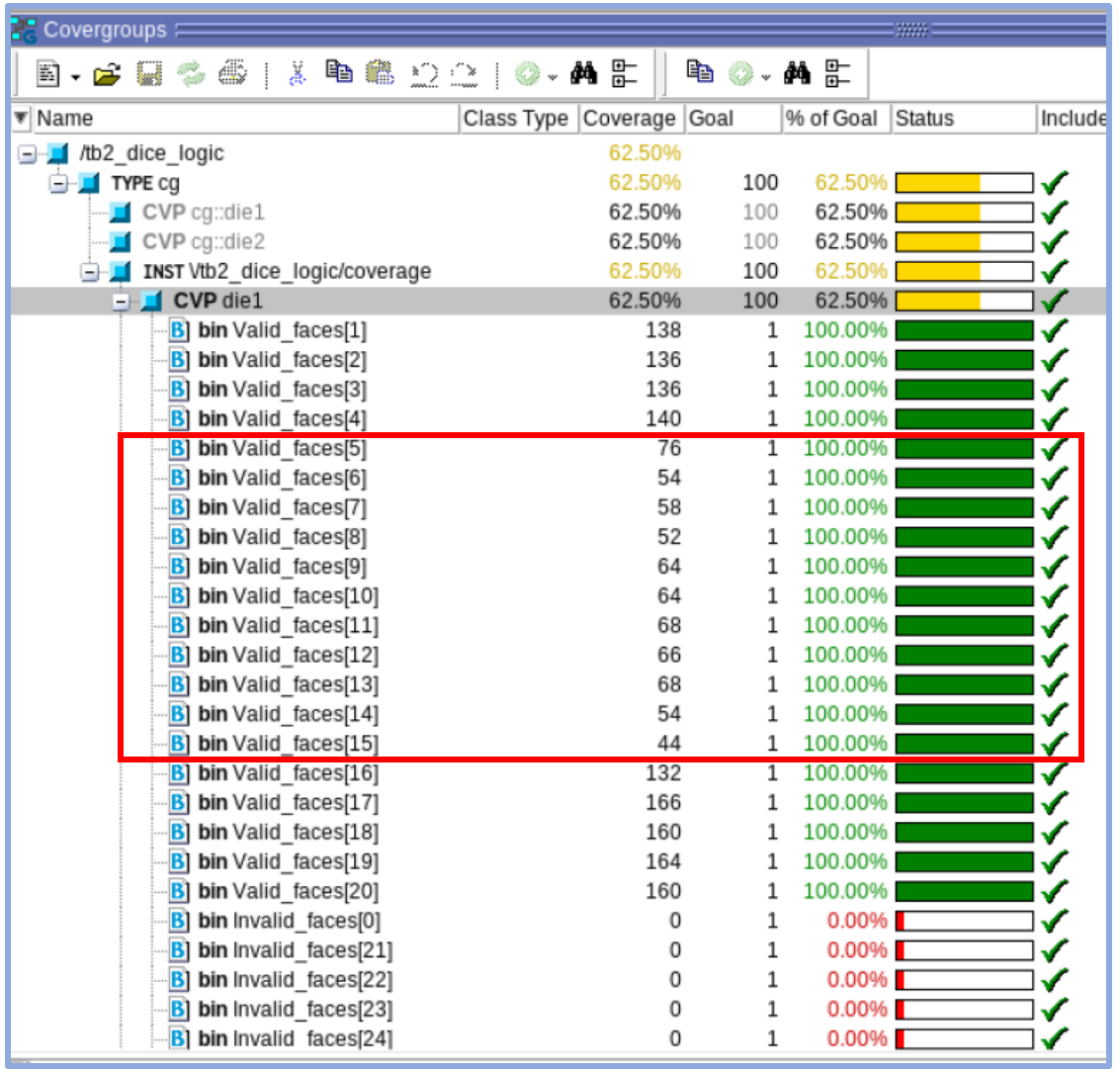
Specifies **weighted or proportional probability** for a random variable.

```
constraint constraint_name {
   variable dist {
      value1 := weight1,
      value2 := weight2,
      [range1_start : range1_end] := weight3,
      [range2_start : range2_end] :/ weight4
   };
}
```

(Example from dice_tb2.sv)

```
constraint weighted_c {
   die1_val dist {[1:4] := 7, [5:15] := 3, [16:20] := 7};
   die2_val dist {[1:4] := 7, [5:15] := 3, [16:20] := 7};
 }
```

dice_tb2.sv Covergroup Window
(repeat 1000 times)



unique Keyword in Constraints

The **generated** values for a random variable **must be unique.**

```
constraint constraint_name {
   unique { var1, var2, var3, ... };
}
```

(Example from dice_tb3.sv)

```
//  dice values must be different
    constraint unique_c {
        unique {die1_val, die2_val};
    }
```

dice_tb3.sv
Transcript Window

```
# DIE1=3 DIE2=16 SUM=19 DOUBLE=0 ODD=1 EVEN=0
# DIE1=2 DIE2=1 SUM=3 DOUBLE=0 ODD=1 EVEN=0
# DIE1=17 DIE2=8 SUM=25 DOUBLE=0 ODD=1 EVEN=0
# DIE1=1 DIE2=6 SUM=7 DOUBLE=0 ODD=1 EVEN=0
# DIE1=9 DIE2=4 SUM=13 DOUBLE=0 ODD=1 EVEN=0
# DIE1=8 DIE2=11 SUM=19 DOUBLE=0 ODD=1 EVEN=0
# DIE1=9 DIE2=4 SUM=13 DOUBLE=0 ODD=1 EVEN=0
# DIE1=5 DIE2=4 SUM=9 DOUBLE=0 ODD=1 EVEN=0
# DIE1=6 DIE2=1 SUM=7 DOUBLE=0 ODD=1 EVEN=0
# DIE1=4 DIE2=11 SUM=15 DOUBLE=0 ODD=1 EVEN=0
# DIE1=13 DIE2=6 SUM=19 DOUBLE=0 ODD=1 EVEN=0
# DIE1=6 DIE2=15 SUM=21 DOUBLE=0 ODD=1 EVEN=0
# DIE1=4 DIE2=3 SUM=7 DOUBLE=0 ODD=1 EVEN=0
# DIE1=9 DIE2=14 SUM=23 DOUBLE=0 ODD=1 EVEN=0
# DIE1=20 DIE2=9 SUM=29 DOUBLE=0 ODD=1 EVEN=0
# DIE1=7 DIE2=14 SUM=21 DOUBLE=0 ODD=1 EVEN=0
# DIE1=1 DIE2=12 SUM=13 DOUBLE=0 ODD=1 EVEN=0
# DIE1=14 DIE2=7 SUM=21 DOUBLE=0 ODD=1 EVEN=0
# DIE1=4 DIE2=11 SUM=15 DOUBLE=0 ODD=1 EVEN=0
# DIE1=6 DIE2=13 SUM=19 DOUBLE=0 ODD=1 EVEN=0
# DIE1=8 DIE2=11 SUM=19 DOUBLE=0 ODD=1 EVEN=0
# DIE1=7 DIE2=2 SUM=9 DOUBLE=0 ODD=1 EVEN=0
```

if-else Keyword in Constraints

Creates an **if-then-else conditional** relationship among variables.

```
constraint constraint_name {
   if (condition)
      <constraint_statement1>;
   else
      <constraint_statement2>;
}
```

(Example from dice_tb3.sv)

```
constraint ifelse_c {
     if (die1_val % 2 == 0)
           die2_val % 2 == 1;
     else
           die2_val % 2 == 0;
 }
```

RED: even
Highlighted: odd

dice_tb3.sv
Transcript Window

```
# DIE1=3 DIE2=16 SUM=19 DOUBLE=0 ODD=1 EVEN=0
# DIE1=2 DIE2=1 SUM=3 DOUBLE=0 ODD=1 EVEN=0
# DIE1=17 DIE2=8 SUM=25 DOUBLE=0 ODD=1 EVEN=0
# DIE1=1 DIE2=6 SUM=7 DOUBLE=0 ODD=1 EVEN=0
# DIE1=9 DIE2=4 SUM=13 DOUBLE=0 ODD=1 EVEN=0
# DIE1=8 DIE2=11 SUM=19 DOUBLE=0 ODD=1 EVEN=0
# DIE1=9 DIE2=4 SUM=13 DOUBLE=0 ODD=1 EVEN=0
# DIE1=5 DIE2=4 SUM=9 DOUBLE=0 ODD=1 EVEN=0
# DIE1=6 DIE2=1 SUM=7 DOUBLE=0 ODD=1 EVEN=0
# DIE1=4 DIE2=11 SUM=15 DOUBLE=0 ODD=1 EVEN=0
# DIE1=13 DIE2=6 SUM=19 DOUBLE=0 ODD=1 EVEN=0
# DIE1=6 DIE2=15 SUM=21 DOUBLE=0 ODD=1 EVEN=0
# DIE1=4 DIE2=3 SUM=7 DOUBLE=0 ODD=1 EVEN=0
# DIE1=9 DIE2=14 SUM=23 DOUBLE=0 ODD=1 EVEN=0
# DIE1=20 DIE2=9 SUM=29 DOUBLE=0 ODD=1 EVEN=0
# DIE1=7 DIE2=14 SUM=21 DOUBLE=0 ODD=1 EVEN=0
# DIE1=1 DIE2=12 SUM=13 DOUBLE=0 ODD=1 EVEN=0
# DIE1=14 DIE2=7 SUM=21 DOUBLE=0 ODD=1 EVEN=0
# DIE1=4 DIE2=11 SUM=15 DOUBLE=0 ODD=1 EVEN=0
```

solve-before Keyword in Constraints

Use to generate one variable **before another**, when one variable's value **depends on another**

```
constraint <constraint_name> {
   solve <var1> before <var2>;
}
```

(Example from dice_tb4.sv)

```
// Force solver to generate die1 first, then die2.
// Ensures die2 depends on generate die1 value.
    constraint order_c {
       solve die1_val before die2_val;  }

// Dependency: die2 must be within ±3 of die1"
    constraint dependent_c {
       die2_val inside { [die1_val-3 : die1_val+3] };
    }
```

dice_tb4.sv
Transcript Window

```
#  Rolling normally. Last sum = 0
#  Rolled DIE1=7 DIE2=6 SUM=13
# ========================================
# DIE1=7 DIE2=6 SUM=13 DOUBLE=0 ODD=1 EVEN=0
#  Rolling normally. Last sum = 13
#  Rolled DIE1=7 DIE2=9 SUM=16
# ========================================
# DIE1=7 DIE2=9 SUM=16 DOUBLE=0 ODD=0 EVEN=1
#  Rolling normally. Last sum = 16
#  Rolled DIE1=14 DIE2=13 SUM=27
# ========================================
# DIE1=14 DIE2=13 SUM=27 DOUBLE=0 ODD=1 EVEN=0
#  Rolling normally. Last sum = 27
#  Rolled DIE1=7 DIE2=4 SUM=11
# ========================================
# DIE1=7 DIE2=4 SUM=11 DOUBLE=0 ODD=1 EVEN=0
#  Rolling normally. Last sum = 11
#  Rolled DIE1=15 DIE2=14 SUM=29
# ========================================
# DIE1=15 DIE2=14 SUM=29 DOUBLE=0 ODD=1 EVEN=0
#  Rolling normally. Last sum = 29
#  Rolled DIE1=11 DIE2=14 SUM=25
# ========================================
# DIE1=11 DIE2=14 SUM=25 DOUBLE=0 ODD=1 EVEN=0
#  Rolling normally. Last sum = 25
#  Rolled DIE1=9 DIE2=7 SUM=16
```

soft Keyword in Constraints

A **soft constraint** will be applied **only if no** other "regular" constraint **overrides it**.

```
constraint <constraint_name> {
   soft <expression>;
}
```

(Example from dice_tb4.sv) * (To be explained next)

```
constraint range_c {
  soft die1_val inside {[1:20]};    // can be overridden
  soft die2_val inside {[1:20]};    // can be overridden
 }
...
repeat (100) begin
   assert(stim.randomize() with { die1_val inside{[21:30]};})
                  // inline constraint overrides only die1_val
```

dice_tb4soft.sv Transcript Window

```
*  # Rolling normally. Last sum = 0
   # Rolled DIE1=22 DIE2=20 SUM=42
   # ========================================
   # DIE1=22 DIE2=20 SUM=42 DOUBLE=0 ODD=0 EVEN=1
*  # Last sum (42) too high! Will set die2 to one, after roll.
*  # die2 set to 1 because last sum was high
   # Rolled DIE1=22 DIE2=1 SUM=23
   # ========================================
   # DIE1=22 DIE2=1 SUM=23 DOUBLE=0 ODD=1 EVEN=0
*  # Rolling normally. Last sum = 23
   # Rolled DIE1=23 DIE2=20 SUM=43
   # ========================================
   # DIE1=23 DIE2=20 SUM=43 DOUBLE=0 ODD=1 EVEN=0
   # Last sum (43) too high! Will set die2 to one, after roll.
   # die2 set to 1 because last sum was high
   # Rolled DIE1=22 DIE2=1 SUM=23
   # ========================================
   # DIE1=22 DIE2=1 SUM=23 DOUBLE=0 ODD=1 EVEN=0
   # Rolling normally. Last sum = 23
   # Rolled DIE1=21 DIE2=18 SUM=39
   # ========================================
   # DIE1=21 DIE2=18 SUM=39 DOUBLE=0 ODD=1 EVEN=0
   # Last sum (39) too high! Will set die2 to one, after roll.
   # die2 set to 1 because last sum was high
   # Rolled DIE1=23 DIE2=1 SUM=24
   # ========================================
   # DIE1=23 DIE2=1 SUM=24 DOUBLE=0 ODD=0 EVEN=1
   # Rolling normally. Last sum = 24
```

pre_randomize() Function

- A **built-in function** which is **automatically** invoked just **before** the actual randomization process (e.g., after assert(stim.randomize() ...))
- Allows designer to add code or set up variables before randomization

(Example from dice_tb4.sv)

```
// pre_randomize, display if last sum was too high or normal
function void pre_randomize();
    if (last_sum > 30) begin
       $display(" Last sum (%0d) too high! Will set die2 to one, after roll.", last_sum);
    end
    else begin
      $display(" Rolling normally. Last sum = %0d", last_sum);
    end
endfunction
```

pre_randomize() Function

- A **built-in function** which is **automatically** randomization process (e.g., after assert(st
- Allows designer to add code or set up varia

(Example from dice_tb4.sv)

```
// pre_randomize, display if last sum was too high c
function void pre_randomize();
    if (last_sum > 30) begin
      $display(" Last sum (%0d) too high! Will set c
    end
    else begin
      $display(" Rolling normally. Last sum = %0d",
    end
endfunction
```

dice_tb4solf.sv Transcript Window

```
#  Rolling normally. Last sum = 0
#  Rolled DIE1=22 DIE2=20 SUM=42
# ========================================
# DIE1=22 DIE2=20 SUM=42 DOUBLE=0 ODD=0 EVEN=1
#  Last sum (42) too high! Will set die2 to one, after roll.
#  die2 set to 1 because last sum was high
#  Rolled DIE1=22 DIE2=1 SUM=23
# ========================================
# DIE1=22 DIE2=1 SUM=23 DOUBLE=0 ODD=1 EVEN=0
#  Rolling normally. Last sum = 23
#  Rolled DIE1=23 DIE2=20 SUM=43
# ========================================
# DIE1=23 DIE2=20 SUM=43 DOUBLE=0 ODD=1 EVEN=0
#  Last sum (43) too high! Will set die2 to one, after roll.
#  die2 set to 1 because last sum was high
#  Rolled DIE1=22 DIE2=1 SUM=23
# ========================================
# DIE1=22 DIE2=1 SUM=23 DOUBLE=0 ODD=1 EVEN=0
#  Rolling normally. Last sum = 23
#  Rolled DIE1=21 DIE2=18 SUM=39
# ========================================
# DIE1=21 DIE2=18 SUM=39 DOUBLE=0 ODD=1 EVEN=0
#  Last sum (39) too high! Will set die2 to one, after roll.
#  die2 set to 1 because last sum was high
#  Rolled DIE1=23 DIE2=1 SUM=24
# ========================================
# DIE1=23 DIE2=1 SUM=24 DOUBLE=0 ODD=0 EVEN=1
#  Rolling normally. Last sum = 24
```

post_randomize() Built-in Function

- A **built-in function** which is **automatically** invoked just **after** the actual randomization process (e.g., after assert(stim.randomize() ...)
- Allows designer to add code or set up variables after randomization

(Example from dice_tb4.sv)

```
// post_randomize(), decide if to override  any randomly generated value
function void post_randomize();
   if (last_sum > 30) begin
     die2_val = 1;  // actually changed the generated value to 1
     $display(" die2 set to 1 because last sum was high");
   end
   last_sum = die1_val + die2_val;
     $display(" Rolled DIE1=%0d DIE2=%0d SUM=%0d", die1_val, die2_val, last_sum);
     $display("=========================================");
 endfunction
```

post_randomize() Built-in Fu

- A **built-in function** which is **automatically** randomization process (e.g., after assert(st
- Allows designer to add code or set up varia

(Example from dice_tb4.sv)

```
// post_randomize(), decide if to override any ran
function void post_randomize();
   if (last_sum > 30) begin
    die2_val = 1;  // actually changed the generate
    $display(" die2 set to 1 because last sum was l
   end
   last_sum = die1_val + die2_val;
    $display(" Rolled DIE1=%0d DIE2=%0d SUM=%
    $display("==========
 endfunction
```

dice_tb4solf.sv Transcript Window

```
#  Rolling normally. Last sum = 0
#  Rolled DIE1=22 DIE2=20 SUM=42
# ========================================
# DIE1=22 DIE2=20 SUM=42 DOUBLE=0 ODD=0 EVEN=1
#  Last sum (42) too high! Will set die2 to one, after roll.
#  die2 set to 1 because last sum was high
#  Rolled DIE1=22 DIE2=1 SUM=23
# ========================================
# DIE1=22 DIE2=1 SUM=23 DOUBLE=0 ODD=1 EVEN=0
#  Rolling normally. Last sum = 23
#  Rolled DIE1=23 DIE2=20 SUM=43
# ========================================
# DIE1=23 DIE2=20 SUM=43 DOUBLE=0 ODD=1 EVEN=0
# Last sum (43) too high! Will set die2 to one, after roll.
#  die2 set to 1 because last sum was high
#  Rolled DIE1=22 DIE2=1 SUM=23
# ========================================
# DIE1=22 DIE2=1 SUM=23 DOUBLE=0 ODD=1 EVEN=0
#  Rolling normally. Last sum = 23
#  Rolled DIE1=21 DIE2=18 SUM=39
# ========================================
# DIE1=21 DIE2=18 SUM=39 DOUBLE=0 ODD=1 EVEN=0
#  Last sum (39) too high! Will set die2 to one, after roll.
#  die2 set to 1 because last sum was high
#  Rolled DIE1=23 DIE2=1 SUM=24
# ========================================
# DIE1=23 DIE2=1 SUM=24 DOUBLE=0 ODD=0 EVEN=1
#  Rolling normally. Last sum = 24
```